formatters out of the box, but as always, it’s easy to add additional ones or change the way the defaults work.

20.5.1 Customizing the default formatters: Adding XML support

As with most of ASP.NET Core, the Web API formatters are completely customizable. By default, only formatters for plain text (`text/plain`), HTML (`text/html`), and JSON (`application/json`) are configured. Given the common use case of single-page application (SPAs) and mobile applications, this will get you a long way. But sometimes you need to be able to return data in a different format, such as XML.

Newtonsoft.Json vs. System.Text.Json

Newtonsoft.Json, also known as Json.NET, has for a long time been the canonical way to work with JSON in .NET. It’s compatible with every version of .NET under the sun, and it will no doubt be familiar to virtually all .NET developers. Its reach was so great that even ASP.NET Core took a dependency on it!

That all changed with the introduction of a new library in ASP.NET Core 3.0, System .Text.Json, which focuses on performance. In .NET Core 3.0 onward, ASP.NET Core uses System.Text.Json by default instead of Newtonsoft.Json.

The main difference between the libraries is that System.Text.Json is picky about its JSON. It will generally only deserialize JSON that matches its expectations. For example, System.Text.Json won’t deserialize JSON that uses single quotes around strings; you have to use double quotes.

If you’re creating a new application, this is generally not a problem; you quickly learn to generate the correct JSON. But if you’re converting an application to ASP.NET Core or are sending JSON to a third party you don’t control, these limitations can be real stumbling blocks.